

## Festival

Mr. Malnar više nije onaj gurman što je znao progutati sendvič za tri osobe bez da trepne. Danas, nakon kobnog pada s bicikla, terapiju je zamijenio koktelima u *Pueblo Chico*. I unatoč preporukama doktorice, odlučio je i dalje ne uzeti godišnji pa se tako nakon Bolivije sprema za turneju po Amerikama: Buenos Aires, Bogota, pa New York... neka nova lica, nova pića i vjerojatno još jedna što će mu ostati u kontaktima pod imenom "ne javljaj se".

Ali čovjek nauči s vremenom da srce ne možeš stalno lomiti. Zato je Mr. Malnar, uvijek majstor prilagodbe sa smislom za stil, pronašao elegantno rješenje: kratki aranžmani, plaćeni unaprijed, na R1 račun i osmijeh koji ne traži obećanja. Latinke su, kaže, posebna priča; plamteće i dovoljno pametne da ne postavljaju nepotrebna pitanja. "Nije to ljubav," zna reći, "ali dođe na vrijeme i ode bez scene." A to je skoro pa romantika.

Jedna od njih je i Nayra. Mr. Malnar joj je odlučio zadati igru, a ako se pokaže dovoljno pametna, povest će ju na turneju kao "+1". Igra se sastoji od korištenja žetona za kupnju kupona. Kupnjom kupona umnožava se njezin broj žetona. Cilj je dobiti što više kupona.

Ona započinje igru s žetonima  $A$ . Postoje kuponi u vrijednosti  $N$ , numerirani od 0 do  $N - 1$ . Nayra mora platiti  $P[i]$  žetona ( $0 \leq i < N$ ) kako bi kupila kupon  $i$  (i mora imati barem  $P[i]$  žetona prije kupnje).

Štoviše, svakom kuponu  $i$  ( $0 \leq i < N$ ) dodjeljuje se **tip**, označeno s  $T[i]$ , što je cijeli broj između 1 i 4, **uključivo**. Nakon što Nayra kupi kupon  $i$ , Preostali broj žetona koje ima množi se s  $T[i]$ . Formalno, ako u nekom trenutku igre ima  $X$  žetona, i kupuje kupon  $i$  (što zahtijeva  $X \geq P[i]$ ), onda će nakon kupnje imati  $(X - P[i]) \cdot T[i]$  žetona.

Vaš je zadatak odrediti koje kupone Nayra treba kupiti i kojim redoslijedom, kako bi maksimizirala ukupan broj **kupona** koje ima na kraju. Ako postoji više od jednog niza kupnji kojima se to postiže, možete prijaviti bilo koju od njih.

## Implementation Details

You should implement the following procedure.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- $A$ : the initial number of tokens Nayra has.
- $P$ : an array of length  $N$  specifying the prices of the coupons.
- $T$ : an array of length  $N$  specifying the types of the coupons.
- This procedure is called exactly once for each test case.

The procedure should return an array  $R$ , which specifies Nayra's purchases as follows:

- The length of  $R$  should be the maximum number of coupons she can buy.
- The elements of the array are the indices of the coupons she should buy, in chronological order. That is, she buys coupon  $R[0]$  first, coupon  $R[1]$  second, and so on.
- All elements of  $R$  must be unique.

If no coupons can be bought,  $R$  should be an empty array.

## Constraints

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$  for each  $i$  such that  $0 \leq i < N$ .
- $1 \leq T[i] \leq 4$  for each  $i$  such that  $0 \leq i < N$ .

## Subtasks

| Subtask | Score | Additional Constraints                                                |
|---------|-------|-----------------------------------------------------------------------|
| 1       | 5     | $T[i] = 1$ for each $i$ such that $0 \leq i < N$ .                    |
| 2       | 7     | $N \leq 3000$ ; $T[i] \leq 2$ for each $i$ such that $0 \leq i < N$ . |
| 3       | 12    | $T[i] \leq 2$ for each $i$ such that $0 \leq i < N$ .                 |
| 4       | 15    | $N \leq 70$                                                           |
| 5       | 27    | Nayra can buy all $N$ coupons (in some order).                        |
| 6       | 16    | $(A - P[i]) \cdot T[i] < A$ for each $i$ such that $0 \leq i < N$ .   |
| 7       | 18    | No additional constraints.                                            |

## Examples

### Example 1

Consider the following call.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra initially has  $A = 13$  tokens. She can buy 3 coupons in the order shown below:

| Coupon bought | Coupon price | Coupon type | Number of tokens after the purchase |
|---------------|--------------|-------------|-------------------------------------|
| 2             | 8            | 3           | $(13 - 8) \cdot 3 = 15$             |
| 3             | 14           | 4           | $(15 - 14) \cdot 4 = 4$             |
| 0             | 4            | 1           | $(4 - 4) \cdot 1 = 0$               |

In this example, it is not possible for Nayra to buy more than 3 coupons, and the sequence of purchases described above is the only way in which she can buy 3 of them. Hence, the procedure should return  $[2, 3, 0]$ .

## Example 2

Consider the following call.

```
max_coupons(9, [6, 5], [2, 3])
```

In this example, Nayra can buy both coupons in any order. Hence, the procedure should return either  $[0, 1]$  or  $[1, 0]$ .

## Example 3

Consider the following call.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

In this example, Nayra has one token, which is not enough to buy any coupons. Hence, the procedure should return  $[]$  (an empty array).

## Sample Grader

Input format:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Output format:

```
S  
R[0] R[1] ... R[S-1]
```

Here,  $S$  is the length of the array  $R$  returned by `max_coupons`.